

# [BE] Study Jam 2: Menyambungkan Projek dengan Database PostgreSQL

## 1. Persiapan

Untuk mengikuti pembelajaran ini kamu perlu menyiapkan:

- Bahasa **Go Lang** (<https://go.dev/>).
- **Git** untuk clone starter proyek (<https://git-scm.com/>).
- **VS Code** untuk text editor, atau editor kesukaanmu (<https://code.visualstudio.com/>). Pastikan menginstall Plugin Go Lang.
- **Postman** untuk test API yang kita buat nanti (<https://www.postman.com/>).

## 2. Setup Database PostgreSQL

Untuk mempermudah pembelajaran, kita akan menggunakan **Neon DB** sebagai database PostgreSQL online. Ini akan membantu kita menghindari proses instalasi PostgreSQL di mesin lokal yang bisa memakan waktu.

1. Kunjungi website Neon DB di <https://neon.tech>
2. Daftar akun baru menggunakan akun GitHub atau email
3. Setelah login, Buat projek baru, isikan nama projek dan juga nama database yang akan kita gunakan.

 / **Get started with Neon for Free**

Project name

Simple Book Collection

Postgres version

17

Database name

simple\_book\_collection

Cloud Service Provider

☐ AWS

☒ Azure

Region

East US 2 (Virginia)

Select the region closest to your application.

Advanced options

Region

East US 2 (Virginia)

Your free plan includes:

0.5 GB storage

24/7 database at 0.25 CU

Autoscaling up to 2 CU

10 branches per project

Create project

#### 4. Klik "Create Project"

Setelah project dibuat, Neon akan memberikan informasi koneksi database yang akan kita gunakan nanti, termasuk:

- Connection string
- Database name
- Username
- Password

Simpan informasi ini karena akan digunakan untuk mengonfigurasi koneksi database di aplikasi Go kita nanti.

⚠ Jangan pernah membagikan credential database Anda kepada siapapun dan jangan commit credential ke repository Git.

### 3. GORM

GORM adalah ORM (Object Relational Mapping) library yang populer untuk bahasa Go. GORM memungkinkan kita untuk berinteraksi dengan database menggunakan struct Go, sehingga kita tidak perlu menulis query SQL secara manual.

Beberapa fitur utama GORM:

- Auto Migration - Membuat/mengupdate struktur tabel berdasarkan struct
- Associations - Mendukung relasi one-to-one, one-to-many, many-to-many
- Hooks - Callbacks before/after operasi database
- Preloading - Eager loading relasi
- Transactions - Mendukung database transactions

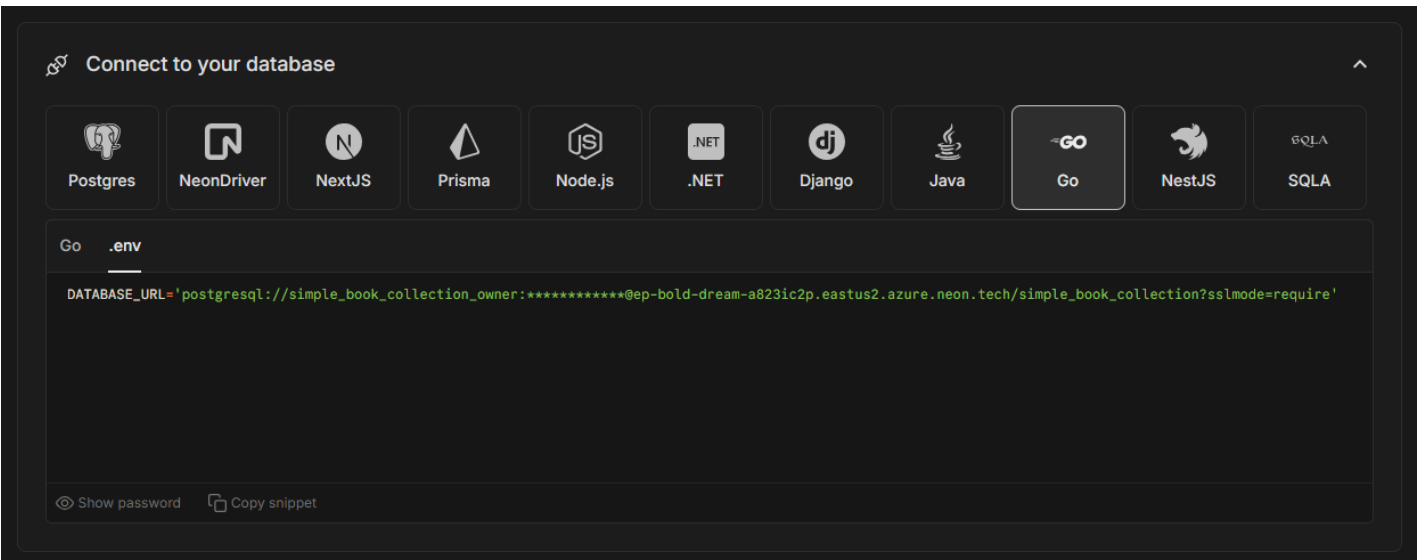
Untuk menambahkan GORM ke proyek Go, jalankan perintah berikut di terminal:

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres
```

Library pertama adalah GORM itu sendiri, sedangkan yang kedua adalah driver PostgreSQL untuk GORM karena kita akan menggunakan PostgreSQL sebagai database.

### 4. Sambungkan Database dengan Proyek Golang Kita

1. Di bagian quickstart, pilih bahasa **Go** lalu bagian `.env` , copy isinya lalu letakkan di file `.env` di proyek kita.



2. Untuk bisa memakai file `.env` kita perlu menambahkan satu library lagi yaitu `godotenv` . Jalankan perintah berikut di terminal:

```
go get -u github.com/joho/godotenv
```

Lalu tambahkan line berikut di `main.go` :

```
err := godotenv.Load(".env")
if err != nil {
    log.Fatalln(".env is not load properly")
}
```

3. Buat folder DB dan file `db.go` untuk menyambungkan ke database.

```
package db

import (
    "os"
```

```

    "time"

    "github.com/anjarmath/01_golang_api_sederhana/model"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

var DB *gorm.DB

// Connnect to postgresql database
func InitDB() {
    conn_url := os.Getenv("DATABASE_URL")

    database, err := gorm.Open(postgres.Open(conn_url), &gorm.Config{})
    database.Exec(`CREATE EXTENSION IF NOT EXISTS "uuid-oss";`)
    if err != nil {
        panic("Cannot connect to database")
    }

    // Auto Migrate digunakan untuk membuat table secara otomatis di database
    database.AutoMigrate(
        &model.Book{},
    )

    db, err := database.DB()
    if err != nil {
        panic("Cant connect to database")
    }
    db.SetMaxIdleConns(10)
    db.SetMaxOpenConns(100)
    db.SetConnMaxLifetime(time.Hour)
    DB = database
}

```

4. Kita juga perlu update struct buku menjadi seperti berikut:

```

type Book struct {
    Id          uuid.UUID `gorm:"type:uuid;default:uuid_generate_v4();primaryKey" json:"id"`
    Judul       string    `json:"judul"`
    Deskripsi    string    `json:"deskripsi"`
    Harga       int       `json:"harga"`
    CreatedAt   time.Time `gorm:"default:now();" json:"created_at"`
    UpdatedAt   *time.Time `json:"updated_at"`
}

```

5. Lalu panggil fungsi `InitDB()` di `main.go` :

```

// Sambungkan ke database
db.InitDB()

```

6. Jalankan proyeknya.

## 5. Tambahkan Fitur Get, Update, dan Delete di Repository

1. Ubah `book.repository.go` menjadi seperti berikut:

```

type BookRepository interface {
    GetBooks() ([]model.Book, error)
    GetBookById(id string) (*model.Book, error)
}

```

```

    AddBook(book *model.Book) error
    UpdateBook(id string, book *model.Book) error
    DeleteBook(id string) error
}

```

2. Kita juga perlu merubah `book.repository.impl.go` yang sudah ada menyesuaikan interface yang baru supaya tidak ada error:

```

// DeleteBook implements BookRepository.
func (b bookRepositoryImpl) DeleteBook(id string) error {
    panic("unimplemented")
}

// GetBookById implements BookRepository.
func (b bookRepositoryImpl) GetBookById(id string) (*model.Book, error) {
    panic("unimplemented")
}

// GetBooks implements BookRepository.
func (b bookRepositoryImpl) GetBooks() ([]model.Book, error) {
    return &model.BooksCollection, nil
}

// UpdateBook implements BookRepository.
func (b bookRepositoryImpl) UpdateBook(id string, book *model.Book) error {
    panic("unimplemented")
}

// AddBook implements BookRepository.
func (b bookRepositoryImpl) AddBook(book *model.Book) error {
    model.BooksCollection = append(model.BooksCollection, *book)
    return nil
}

```

3. Selanjutnya, kita buat implementasi repository baru yang khusus melakukan interaksi data dengan database, di sini kita namai sebagai `book.repository.postgres.go`:

```

package repository

import (
    db "github.com/anjarmath/01_golang_api_sederhana/DB"
    "github.com/anjarmath/01_golang_api_sederhana/model"
)

type bookRepositoryImplPostgres struct{}

// AddBook implements BookRepository.
func (b bookRepositoryImplPostgres) AddBook(book *model.Book) error {
    err := db.DB.Create(book).Error
    return err
}

// DeleteBook implements BookRepository.
func (b bookRepositoryImplPostgres) DeleteBook(id string) error {
    err := db.DB.Delete(&model.Book{}, "id = ?", id).Error
    return err
}

// GetBookById implements BookRepository.

```

```

func (b bookRepositoryImplPostgres) GetBookById(id string) (*model.Book, error) {
    book := new(model.Book)
    if err := db.DB.First(book, "id = ?", id).Error; err != nil {
        return nil, err
    }
    return book, nil
}

// GetBooks implements BookRepository.
func (b bookRepositoryImplPostgres) GetBooks() ([]model.Book, error) {
    book := new([]model.Book)
    if err := db.DB.Find(&book).Error; err != nil {
        return nil, err
    }
    return book, nil
}

// UpdateBook implements BookRepository.
func (b bookRepositoryImplPostgres) UpdateBook(id string, book *model.Book) error {
    err := db.DB.Model(&model.Book{}).Where("id = ?", id).Updates(book).Error
    return err
}

func NewBookRepositoryPostgres() BookRepository {
    return bookRepositoryImplPostgres{}
}

```

## 6. Tambahkan Fitur Get, Update, dan Delete di Controller dan Route

1. Ubah `book.repository.go` :

```

package controller

import (
    "github.com/anjarmath/01_golang_api_sederhana/model"
    "github.com/anjarmath/01_golang_api_sederhana/repository"
    "github.com/gofiber/fiber/v2"
)

type bookController struct {
    bookRepository repository.BookRepository
}

func NewBookController(repo repository.BookRepository) bookController {
    return bookController{
        bookRepository: repo,
    }
}

func (bc bookController) GetBook(c *fiber.Ctx) error {
    books, err := bc.bookRepository.GetBooks()
    if err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: "Terjadi kesalahan",
        })
    }
    return c.JSON(books)
}

```

```

func (bc bookController) GetBookById(c *fiber.Ctx) error {
    id := c.Params("id")
    book, err := bc.bookRepository.GetBookById(id)
    if err != nil {
        return c.Status(fiber.StatusNotFound).JSON(fiber.Error{
            Message: "Tidak ditemukan data",
        })
    }
    return c.JSON(book)
}

func (bc bookController) UpdateBook(c *fiber.Ctx) error {
    id := c.Params("id")
    reqBook := new(model.Book)

    // Cek request body
    if err := c.BodyParser(reqBook); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Code:    fiber.StatusBadRequest,
            Message: "Mohon isikan data dengan benar",
        })
    }

    err := bc.bookRepository.UpdateBook(id, reqBook)
    if err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: "Terjadi kesalahan",
        })
    }
    return c.JSON(fiber.Map{
        "message": "Buku berhasil diubah",
        "buku":    reqBook,
    })
}

func (bc bookController) DeleteBook(c *fiber.Ctx) error {
    id := c.Params("id")
    err := bc.bookRepository.DeleteBook(id)
    if err != nil {
        return c.Status(fiber.StatusNotFound).JSON(fiber.Error{
            Message: "Tidak ditemukan data",
        })
    }
    return c.JSON(fiber.Map{
        "message": "Buku berhasil dihapus",
    })
}

func (bc bookController) AddBook(c *fiber.Ctx) error {
    book := new(model.Book)

    if err := c.BodyParser(book); err != nil {
        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Code:    fiber.StatusBadRequest,
            Message: "Mohon isikan data dengan benar",
        })
    }

    valid := validateBook(book)
    if !valid {

```

```

        return c.Status(fiber.StatusBadRequest).JSON(fiber.Error{
            Message: "Mohon isikan data dengan benar",
        })
    }

    err := bc.bookRepository.AddBook(book)
    if err != nil {
        return c.Status(fiber.StatusInternalServerError).JSON(fiber.Error{
            Message: "Terjadi kesalahan",
        })
    }

    return c.JSON(fiber.Map{
        "message": "Buku berhasil ditambahkan",
        "buku":    book,
    })
}

func validateBook(book *model.Book) bool {
    return !(book.Judul == "" || book.Deskripsi == "" || book.Harga == 0)
}

```

## 2. Lalu ubah book route

```

func SetupBookRoute(r *fiber.App) {
    r.Get("/", func(c *fiber.Ctx) error {
        return c.SendString("Belajar bareng API Sederhana!")
    })

    bookRepository := repository.NewBookRepositoryPostgres()
    bookController := controller.NewBookController(bookRepository)

    // Get Buku
    r.Get("/book", bookController.GetBook)

    // Get Buku berdasarkan ID
    r.Get("/book/:id", bookController.GetBookById)

    // Update Buku
    r.Patch("/book/:id", bookController.UpdateBook)

    // Hapus buku
    r.Delete("/book/:id", bookController.DeleteBook)

    // Post Buku
    r.Post("/book", bookController.AddBook)
}

```

## 3. Jalankan projek

# 7. Apa Selanjutnya?

- Autentikasi dan Autorisasi